

INDIVIDUAL PROJECT REPORT

ON

“CampusSphere: Frontend Development, Rolling QR Pass and Deployment for a Unified College Event and Activity-Points Platform”

Submitted in partial fulfillment of the requirement for the award of Degree of Bachelor of Technology in Computer Science & Engineering

Submitted to:



Rajasthan Technical University, Kota (Raj.)

Submitted By:

Govind Jangid

(24EARCS051)

Team Leader, Group Project CampusSphere

Under the guidance of

PROJECT GUIDE

Er. Ram Babu Buri

Department of Computer Science & Engineering



Session: 2026-27

Department of Computer Science & Engineering

**ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR**

CERTIFICATE

This is to certify that the individual project report entitled “**CampusSphere: Frontend Development, Rolling QR Pass and Deployment for a Unified College Event and Activity-Points Platform**”, submitted by **Govind Jangid (24EARCS051)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering of Rajasthan Technical University, Kota**, is a record of the bonafide work carried out by him as a member and Team Leader of the project group (Govind Jangid, Garvita Jain, Ayushi Garg, Ankit Yadav and Anshika Pathak) at **Arya College of Engineering & I.T., Jaipur** during the session 2026-27, under my guidance and supervision.

To the best of my knowledge, the work embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

Date: _____ Place: Jaipur

Er. Ram Babu Buri
Project Guide
Dept. of CSE, ACEIT, Jaipur

Dr. Akhil Pandey
Head of Department
Dept. of CSE, ACEIT, Jaipur

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Er. Ram Babu Buri**, for his continuous guidance, encouragement and valuable suggestions throughout the project. His insistence on clear requirements, a weekly plan and honest review helped me understand how a large software system is planned and delivered by a team.

I sincerely thank **Dr. Akhil Pandey, Head of the Department of Computer Science and Engineering, Arya College of Engineering & I.T.**, for the academic support and the environment provided for this work. I am also grateful to all the faculty members of the **Department of Computer Science and Engineering** for their guidance and cooperation.

I would like to acknowledge the support of my teammates, **Garvita Jain, Ayushi Garg, Ankit Yadav and Anshika Pathak**, with whom I worked on **CampusSphere**. The discussions on interfaces between the frontend, backend and database sections, the joint review of the specification and the shared planning of the 14-week schedule greatly enriched my understanding of team-based software development.

I also thank the faculty coordinators and student leads of the fifteen clubs of Arya College, whose experience with organising events shaped the requirements of the platform.

Finally, I extend my heartfelt gratitude to my family and friends for their constant support, patience and motivation throughout the course of this project.

Govind Jangid
(24EARCS051)
Computer Science & Engineering

ABSTRACT

This report presents my individual contribution to the group project **CampusSphere**, a unified web platform for college events, club treasuries, verified attendance and AICTE activity points at **Arya College of Engineering & I.T., Jaipur**. The project was undertaken by five students over 14 weeks. The work was divided into five sections: two members for the frontend and deployment (Govind Jangid and Ankit Yadav) and three members for the backend and the database (Garvita Jain, Ayushi Garg and Anshika Pathak).

My section is the student-facing frontend, built with React 19, TypeScript, Vite, Tailwind CSS and TanStack Query, together with my share of the deployment and the role of Team Leader. The report describes the design of the authentication screens, club showcase, event discovery, solo and team registration, the My Tickets dashboard with the 30-second rolling QR pass, Razorpay checkout, verified feedback, certificates and activity-points dashboard, the AI recommended feed and chatbot, and the interactive 3D campus twin built with Three.js.

The report explains the technologies and concepts used, such as client-side token handling, server-state caching, progressive web apps, QR rendering with clock-offset correction and WebGL performance techniques. It also gives my week-by-week plan for the 14-week schedule, the dependencies on my teammates' sections, the test plan and the challenges anticipated with their planned solutions.

This is a design and planning report; performance and usability targets quoted in it are goals to be measured during testing and the pilot. The work provides practical experience in modern frontend engineering, contract-first collaboration and the planning and coordination of a team software project.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION1

- 1.1 Project Overview1
- 1.2 Team and Work Division1
- 1.3 My Role and Responsibilities1
- 1.4 Objectives of My Section2
- 1.5 Project Duration and Schedule3
- 1.6 Tools and Collaboration3

CHAPTER 2 TECHNOLOGIES AND CONCEPTS4

- 2.1 React4
- 2.2 TypeScript4
- 2.3 Vite4
- 2.4 Tailwind CSS4
- 2.5 React Router4
- 2.6 TanStack Query5
- 2.7 Context API and Authentication State5
- 2.8 REST Client and Interceptors5
- 2.9 Progressive Web Apps5
- 2.10 QR Code Rendering and Timers5
- 2.11 Razorpay Checkout5
- 2.12 Three.js and React Three Fiber6
- 2.13 WebSocket Client6
- 2.14 Responsive Design and Accessibility6
- 2.15 Testing Tools6
- 2.16 Git, GitHub and Docker6

CHAPTER 3 INDUSTRY APPLICATIONS AND RELEVANCE7

- 3.1 Event Management and Ticketing Platforms7
- 3.2 Anti-Fraud Digital Tickets7
- 3.3 Campus and Education Applications7
- 3.4 Progressive Web Apps in Practice7
- 3.5 3D Digital Twins7
- 3.6 Digital Credentials and Verification7
- 3.7 Digital Payments in India7
- 3.8 Future Scope8

CHAPTER 4 MY WORK: DESIGN AND IMPLEMENTATION APPROACH9

- 4.1 Frontend Architecture9
- 4.2 Project Structure9
- 4.3 Design System and Theming10
- 4.4 Authentication Screens and Session Handling10
- 4.5 Role-Based Navigation and Route Guards10
- 4.6 Club Showcase10
- 4.7 Event Discovery and Detail10
- 4.8 Registration: Solo and Team11
- 4.9 My Tickets and the Rolling QR Pass11
- 4.10 Payment Checkout12
- 4.11 Feedback and Suggestions12
- 4.12 Certificates and Activity Points Dashboard12
- 4.13 AI Recommended Feed and Concierge Chatbot13
- 4.14 3D Campus Digital Twin13
- 4.15 API Integration and Error Handling13
- 4.16 Performance, Accessibility and Responsiveness13
- 4.17 Testing Approach14

- 4.18 Deployment Responsibilities14
- 4.19 Anticipated Challenges and Planned Solutions14
- 4.20 Integration with Teammates15

CHAPTER 5 14-WEEK WORK PLAN16

- 5.1 My Week-by-Week Plan16
- 5.2 Team Overview16
- 5.3 Reporting and Review17

CHAPTER 6 LEARNING OUTCOMES AND SKILLS18

- 6.1 Technical Skills18
- 6.2 Professional Skills18
- 6.3 Leadership and Teamwork18
- 6.4 Problem Solving and Debugging18
- 6.5 Version Control and Communication19

CHAPTER 7 CONCLUSION20

REFERENCES21

CHAPTER 1

INTRODUCTION

1.1 Project Overview

The group project **CampusSphere** is a unified platform for college events, club treasuries, verified attendance and AICTE activity points at Arya College of Engineering & I.T., Jaipur. It replaces the present practice of chat-group announcements, form-based registration, personal UPI collection, paper attendance and static certificates with one authenticated system for Students, Club Admins and the Super Admin (Dean/HOD). Its main features are a directory of fifteen clubs with dedicated treasuries, an event approval workflow, venue booking with a 30-minute buffer, atomic seat reservation, Razorpay payments credited to club ledgers, a rolling 30-second QR pass, a camera-based gate scanner, attendance-gated feedback, SHA-256-sealed certificates and an automated activity-points ledger.

The project was carried out by five students over 14 weeks under the guidance of **Er. Ram Babu Buri**. This individual report describes my own part: the design and planned development of the student-facing frontend and my share of deployment, together with my role as Team Leader in integrating the work of all five sections.

1.2 Team and Work Division

The work was divided into five sections. Two members, Ankit Yadav and I, took the frontend and deployment. Three members, Garvita Jain, Ayushi Garg and Anshika Pathak, took the backend and the database. Table 1.1 shows the division.

Table 1.1: Division of work among the team

Member	Section	Responsibility in brief
Govind Jangid (Team Leader)	Frontend (student experience)	Student screens, rolling QR pass, checkout, certificates and points UI, AI feed, 3D twin; integration lead
Ankit Yadav	Frontend (admin consoles) and deployment	Club and Super Admin consoles, gate scanner PWA, Kanban, Docker, NGINX, CI/CD
Garvita Jain	Backend	Authentication, RBAC, clubs, events, venue buffer engine, audit, security
Ayushi Garg	Backend	Registration, capacity lock, payments, ledger, QR token service, certificate PDF
Anshika Pathak	Database and backend	Schemas, indexes, seeding, transactions, attendance, feedback, seal and points

1.3 My Role and Responsibilities

My role has two parts. As the **frontend developer for the student experience**, I was assigned the following modules:

- application foundation: project setup, design system, theming (dark and light), routing and API client;
- authentication screens and role-based navigation;
- club showcase, event discovery, search, filters and event detail pages;
- registration flow for solo and team events, including team passcode creation and joining;
- the My Tickets dashboard and the anti-screenshot rolling QR pass with countdown ring and holographic watermark;
- Razorpay checkout integration and payment result screens;
- verified feedback form, public suggestion board and upvoting;
- certificate wallet, public verification page, LinkedIn share button and activity-points dashboard;
- the AI recommended feed and the campus concierge chatbot interface;
- the interactive 3D campus digital twin with event beacons.

As **Team Leader** I was responsible for the shared repository and branch rules, the weekly meeting, the interface table between sections, merging and release of the integrated application, and coordination of the documentation and the final demo. For deployment I support Ankit Yadav by preparing the production build of the frontend, its environment configuration and the PWA settings, while he owns Docker, NGINX, TLS and the CI/CD pipeline.

1.4 Objectives of My Section

- To deliver a fast, responsive and accessible student interface that works well on small phones, since most students will use the pass at the venue gate.
- To make the rolling QR pass reliable: it must always show a valid code, tolerate small clock differences and handle poor connectivity gracefully.
- To hide complexity from the user: payment, ticket issue and team joining should be completed in a few taps.

- To keep the frontend strictly untrusted: price, role and status shown on screen are always taken from the server.
- To keep the code type-safe and testable so that changes to the API contract are caught early.
- To provide a visually engaging and performant 3D campus view without hurting low-end devices.

1.5 Project Duration and Schedule

The project is planned over 14 weeks and follows the ten-phase roadmap: foundation and authentication (Weeks 1-2), clubs (Week 3), events and venues (Weeks 4-5), registration and rolling QR (Week 6), payments (Week 7), gate scanner (Week 8), feedback (Week 9), certificates and points (Week 10), AI suite (Week 11), 3D twin and deployment (Week 12), integration testing (Week 13) and pilot with documentation (Week 14). My own week-by-week plan is in Chapter 5.

1.6 Tools and Collaboration

I use Visual Studio Code, Node.js, Git and GitHub with feature branches and pull requests, browser developer tools with device emulation for mobile testing, and Postman or the mock API for contract testing. The team communicates through a weekly meeting and a shared issue board. Every task in the schedule is an issue with an owner and a week label.

CHAPTER 2

TECHNOLOGIES AND CONCEPTS

This chapter explains the technologies that I use for the frontend and for the deployment of the frontend, and the concepts behind them.

2.1 React

React is a JavaScript library for building user interfaces from components. A component is a function that receives properties and returns a description of the interface. When data changes, React updates only the parts of the page that depend on it. CampusSphere uses React 19 with functional components and hooks such as `useState`, `useEffect` and custom hooks that wrap data fetching or timers. This style fits the application, which is a collection of screens that react to live data such as seat counts and the time left on a QR code.

2.2 TypeScript

TypeScript adds static types to JavaScript. I define interfaces for the entities that the API returns, for example `User`, `Club`, `Event`, `Registration`, `Payment`, `Certificate`, and for the standard response envelope. When the API changes, the compiler points out every place that must be updated. This is very valuable in a five-member team where the frontend and backend are developed in parallel.

2.3 Vite

Vite is a build tool that starts instantly in development using native ES modules and bundles for production with Rollup. It offers fast hot module replacement, environment variables through `import.meta.env`, code splitting with dynamic `import()` and plugin support for PWA features. The 3D scene is loaded lazily so that it does not slow down the first page.

2.4 Tailwind CSS

Tailwind CSS is a utility-first framework in which styling is applied with small class names directly on elements. It gives a consistent spacing and colour scale, responsive variants such as `md:` and `lg:`, and a `dark:` variant for the dark theme. Design tokens such as the brand colour, radius and shadows are defined once in the configuration so that all screens remain consistent.

2.5 React Router

React Router maps URLs to pages and supports nested routes, route parameters (for example `/events/:slug`) and redirects. I use it with route guards: a wrapper component checks the role in the authentication context and redirects an unauthorised visitor to the login page or to a not-allowed page.

2.6 TanStack Query

TanStack Query manages server state: it fetches data, caches it, retries failed requests and refreshes stale data in the background. Instead of writing loading flags and manual caches for each screen, I define a query for each resource, such as the event list or my tickets, and mutations for actions such as registration. After a successful registration the related queries are invalidated so that the seat count and ticket list update automatically.

2.7 Context API and Authentication State

The React Context API shares values across the component tree without passing properties through every level. I use an `AuthContext` for the current user, role and access token, and a `ThemeContext` for dark or light mode. The access token is kept in memory and the refresh token lives in an `HttpOnly` cookie that scripts cannot read, so a cross-site scripting bug cannot steal the long-lived credential.

2.8 REST Client and Interceptors

All network calls go through one API client that adds the bearer token, parses the standard envelope (`success`, `status_code`, `data`, `error_code`) and handles errors centrally. When a request fails with 401 because the 15-minute access token has expired, the client calls `POST /auth/refresh`, stores the new token and retries the original request once. If the refresh also fails, the user is logged out.

2.9 Progressive Web Apps

A Progressive Web App uses a web app manifest and a service worker to make a website installable and partly available offline. For CampusSphere the benefits are an installable icon on the home screen, cached application shell for fast start-up and the ability to keep the last loaded ticket details available when the venue network is weak. The camera-based scanner built by Ankit Yadav also relies on the browser media APIs of the PWA environment.

2.10 QR Code Rendering and Timers

A QR code is a two-dimensional barcode that stores a string. The pass screen renders the token string received from the server as a QR image on a canvas. A timer hook calculates the seconds left to the end of the current 30-second window from the server-provided expiry time, and an animated SVG circle shows the remaining time. Because the phone clock may differ from the server clock, the hook corrects for the offset measured from the response time header.

2.11 Razorpay Checkout

Razorpay Checkout is a JavaScript widget that opens a secure payment window for UPI, cards and net banking. The frontend never decides the price. It asks the backend to create an order, passes the returned order identifier and key to the widget, and sends the resulting payment identifier and signature back to the backend for verification. The ticket is shown only after the backend confirms the payment.

2.12 Three.js and React Three Fiber

Three.js is a library for real-time 3D graphics on top of WebGL. React Three Fiber lets me describe the scene as React components: meshes for buildings, lights and a camera with orbit controls. Performance techniques include low-polygon geometry, shared materials, instanced meshes for repeated objects, frame-rate limits on mobile and loading the module only when the user opens the campus page.

2.13 WebSocket Client

A WebSocket keeps a two-way connection open between browser and server. The student interface listens for ticket confirmation events, and the club interface (by Ankit) listens for live attendee counts. The client reconnects automatically with a back-off delay if the connection drops.

2.14 Responsive Design and Accessibility

Responsive design makes one page adapt to phone, tablet and desktop widths. I follow a mobile-first approach with touch targets of at least 44 pixels, readable font sizes and no horizontal scrolling. Accessibility practices include semantic HTML, labels for form fields, visible focus states, sufficient colour contrast in both themes, alternative text for images and keyboard navigation for modals.

2.15 Testing Tools

Unit tests use Vitest and React Testing Library, which render components and check what the user would see and do. End-to-end tests use Playwright, which drives a real browser through full journeys such as register, pay and show the pass. Mock Service Worker can simulate the backend so that frontend tests do not depend on the backend schedule.

2.16 Git, GitHub and Docker

Git records the history of the code and GitHub hosts the shared repository, pull requests and issue board. The frontend is built into static files and packaged in a Docker image with NGINX, which Ankit Yadav orchestrates with the backend and the database. Docker guarantees that the same image runs on a developer laptop, the staging server and the college server.

CHAPTER 3

INDUSTRY APPLICATIONS AND RELEVANCE

3.1 Event Management and Ticketing Platforms

Digital ticketing is now standard for concerts, conferences and sports. Users expect to discover events, pay online and receive a ticket on the phone. CampusSphere adapts this model to a college, with additional features that a public ticketing website does not need, such as club ledgers under a Dean's audit and academic credit.

3.2 Anti-Fraud Digital Tickets

Ticket fraud through copied barcodes is a known problem. Transport and venue applications address it by rotating barcodes that change every few seconds. The same idea in my rolling QR pass shows how a front-end timer, a server-signed token and a scanner validation work together.

3.3 Campus and Education Applications

Universities increasingly use apps for timetables, attendance, fees and student engagement. A unified extracurricular platform that records verified participation supports accreditation and gives students a portfolio, which employers value.

3.4 Progressive Web Apps in Practice

Many large services provide PWAs so that users do not need to install a native application. For a college where students have a wide range of phones and limited storage, a PWA lowers the entry barrier.

3.5 3D Digital Twins

A digital twin is a virtual model of a physical place or system. Industries use twins for factories and smart buildings. A campus twin helps new students find venues, shows where events are happening and demonstrates the use of WebGL in a practical product.

3.6 Digital Credentials and Verification

Digital certificates and badges that can be verified online are increasingly accepted by employers and professional networks. A verification page that anyone can open without logging in makes a student's achievements believable. My certificate wallet and verification page are the user-facing part of this idea.

3.7 Digital Payments in India

UPI has made small digital payments common in India. A campus platform that accepts UPI through a gateway, and credits a club account instead of a person, brings the same convenience with better accountability.

3.8 Future Scope

The frontend can be extended with native mobile applications, push notifications, NFC-assisted entry, multi-language support including Hindi, offline-first ticket storage and extension to inter-college events. The 3D twin could show live crowd density, and the recommender interface could explain why an event is suggested.

CHAPTER 4

MY WORK: DESIGN AND IMPLEMENTATION APPROACH

This chapter describes the design of the modules assigned to me and the way they connect to the work of my teammates. It is based on the project specification suite and on the schedule in Chapter 5.

4.1 Frontend Architecture

The frontend is a single-page application organised in layers (Figure 4.1). Pages are bound to routes, feature components encapsulate each behaviour such as the QR pass or the team modal, and a small UI kit provides common elements. Server data is handled by TanStack Query, shared client state by contexts, and all backend calls pass through one API client. The built application is served by NGINX in a Docker image.

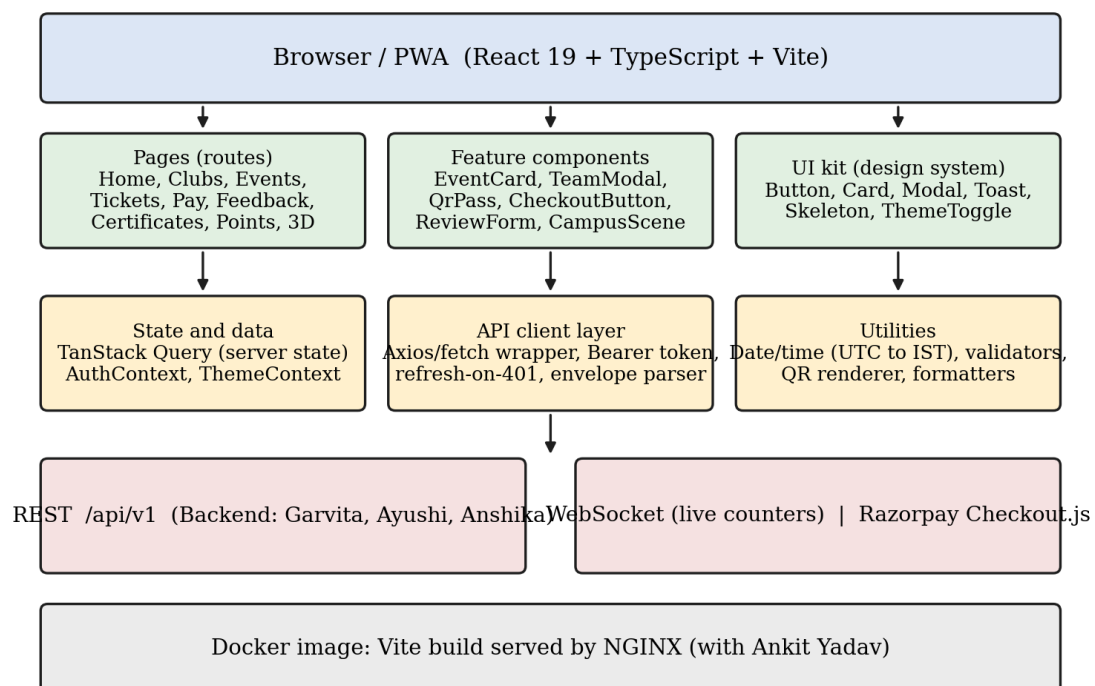


Figure 4.1: Layered frontend architecture of the student application

4.2 Project Structure

```
src/
  api/          client.ts, auth.ts, events.ts, payments.ts, ...
  components/   ui/ (Button, Card, Modal, Toast), layout/
  features/     auth/, clubs/, events/, tickets/, checkout/,
                feedback/, certificates/, points/, ai/, campus3d/
  hooks/        useCountdown, useAuth, useTheme, useSocket
  pages/        Home, Clubs, EventDetail, MyTickets, ...
  types/        user.ts, event.ts, registration.ts, api.ts
```

4.3 Design System and Theming

A small design system keeps the interface consistent. Colour tokens (primary, surface, text, success, warning, danger) are defined as CSS variables with separate values for light and dark themes. Typography uses a clean sans-serif with a clear size scale. Components such as Button, Card, Modal, Toast notification and Skeleton loader are reused everywhere. The theme choice follows the system setting and is saved for the next visit.

4.4 Authentication Screens and Session Handling

The registration form collects name, institutional email, password, roll number (for example 22EACIT089), department, semester and interests. Validation runs in the browser for quick feedback (email domain, password strength, roll format) but is never treated as final, since the backend validates again. After login the access token is stored in memory and the user and role are placed in the AuthContext. A silent refresh on page load restores the session using the HttpOnly cookie. The login page shows clear messages for wrong credentials and for the rate limit of five attempts per minute.

4.5 Role-Based Navigation and Route Guards

Table 4.1: Navigation by role

Role	Main navigation items
Student	Home, Clubs, Events, My Tickets, Points, Certificates, Campus 3D, Concierge
Club Admin	Dashboard, Event wizard, Registrations, Scanner, Treasury, Suggestions Kanban (by Ankit Yadav)
Super Admin	Approvals, Finance audit, Clubs overview, Transcripts, Venue overrides (by Ankit Yadav)

A guard component reads the role from the context. If a student types an administrator URL, the guard redirects to a not-allowed page. This is only a convenience: the real protection is the authorisation check in the backend service layer.

4.6 Club Showcase

The club page lists all fifteen clubs as cards with logo, category chips, faculty coordinator and the number of events and active members. A category filter and a search box narrow the list. A club profile page shows the description, banner, upcoming events, past events and social links. Data comes from `GET /clubs` and is cached, since the list changes rarely.

4.7 Event Discovery and Detail

The event feed supports full-text search, category and tag filters, date range, free or paid, and solo or team. Cards show the banner, title, club, date, venue and a seat indicator. Query parameters are kept in the URL so that a filtered view can be shared. The detail page shows the Markdown description rendered safely, schedule, venue, ticket price, activity points awarded, a live remaining-seats indicator and the register button, whose label changes according to the user's state (Register, Pay, Registered, Sold out or Event ended).

4.8 Registration: Solo and Team

For solo events a single confirmation dialog calls `POST /registrations/solo`. For team events (hackathons and esports) a multi-step modal lets a captain create a team, name it and receive a passcode such as `ACE-789X` that can be copied or shared. Teammates enter the passcode in a join dialog. The interface shows the roster and the minimum and maximum team size (for example two to four) and indicates when the team becomes valid. If the backend answers that the event is sold out, the user sees a waitlist message instead of a generic error.

4.9 My Tickets and the Rolling QR Pass

The My Tickets page lists upcoming and past tickets. Opening a ticket shows the pass screen (Figure 4.2). The screen requests `GET /registrations/:id/dynamic-qr`, which returns a signed token, the validity in seconds and the expiry time. The token is rendered as a QR code and a circular SVG ring animates the remaining time. A short time before the window ends, the client fetches the next token so that the code never stays blank. A shimmering holographic watermark is overlaid on the code, making a still screenshot visibly different from the live screen, and the ticket number, student name and roll number are shown so that the volunteer can compare identity.

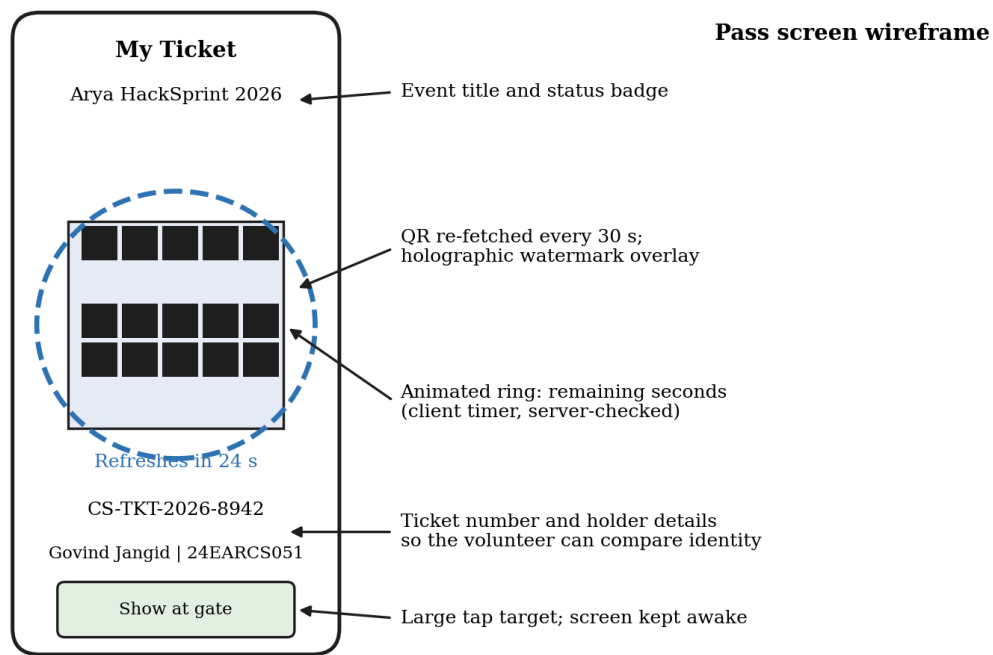


Figure 4.2: Wireframe of the rolling QR pass screen

```
function useCountdown(expiresAtMs, serverOffsetMs) {
  const [left, setLeft] = useState(0);
  useEffect(() => {
    const id = setInterval(() => {
      const now = Date.now() + serverOffsetMs;
      setLeft(Math.max(0, Math.ceil((expiresAtMs - now) / 1000)));
    }, 250);
    return () => clearInterval(id);
  }, [expiresAtMs, serverOffsetMs]);
  return left; // drives ring and triggers next token fetch
}
```

The design handles several edge cases: if the device goes offline, the last token is shown with a clear 'Reconnecting' notice rather than a stale code presented as valid; when the browser tab is in the background, timers are re-synchronised when it returns to the foreground; and the screen is kept awake while the pass is open.

4.10 Payment Checkout

For a paid event the register button starts a three-step flow: create order, pay, verify. The frontend calls `POST /payments/create-order`, opens Razorpay Checkout with the returned order identifier and key, and after payment sends the three returned values to `POST /payments/verify-signature`. While the verification is in progress the user sees a progress indicator and the button is disabled to prevent a second payment. Possible results are handled with distinct screens: success with the ticket, payment failed with a retry, and

pending with a message that the ticket will appear when the gateway confirms the payment through the webhook. The interface never marks a ticket as confirmed on its own.

4.11 Feedback and Suggestions

The feedback form appears on the event page only when the event is COMPLETED. The student rates overall experience and five factors (content depth, organisation, speaker quality, venue facilities, value for time) with star controls, writes an optional review and chooses whether to post anonymously. If the backend returns `VERIFIED_ATTENDANCE_REQUIRED`, the page explains that only students who were scanned at the gate can review. The suggestion board lists ideas sorted by upvotes with a status chip (Submitted, Under review, Planned, Implemented) and an upvote button that updates immediately and rolls back if the server rejects it.

4.12 Certificates and Activity Points Dashboard

The certificate wallet shows each certificate with the event, club, issue date and points, a download button, a copy-link button for the public verification URL and the Add to LinkedIn button which opens a pre-filled certification form. The public verification page, reachable without login, shows a clear Valid or Invalid status with student, event, club and points. The points dashboard shows a progress ring toward 100 points, a bar for each category with its cap, an itemised list of credited events and a button to download the official transcript PDF generated by the backend.

4.13 AI Recommended Feed and Concierge Chatbot

The home page includes a 'Recommended for you' row, using scores from the recommender API, with a short reason such as 'matches your interest in Hackathons'. The concierge chatbot is a floating chat window that sends the user's question (English or Hinglish) to the backend and renders the answer, with links to the events or venues it mentions. Since AI answers can be wrong, the window includes a note and a link to the official event page.

4.14 3D Campus Digital Twin

The campus page renders a low-poly model of the academic blocks, auditorium, innovation hub and sports complex. Buildings hosting live or upcoming events show pulsing neon beacons. Tapping a building moves the camera towards it and opens a drawer with venue details and events, with a register button. For performance, buildings use simple geometries and shared materials, repeated objects are instanced, the render loop pauses when the tab is

hidden, the pixel ratio is limited on phones and a fallback list view is shown if WebGL is unavailable. The target is more than 50 frames per second with an asset bundle under 2 MB.

4.15 API Integration and Error Handling

Every backend error carries a code that the interface maps to a helpful message and action.

Table 4.2: Mapping of API errors to user interface behaviour

Error code or status	Frontend behaviour
401 (token expired)	Silent refresh and retry once; if it fails, redirect to login
403 role not allowed	Show not-allowed page
VENUE_BUFFER_COLLISION (409)	Shown in the admin wizard (by Ankit) with alternative venues
DUPLICATE_CHECK_IN (409)	Shown in the scanner (by Ankit) as a red banner and alarm
VERIFIED_ATTENDANCE_REQUIRED (403)	Explain that only gate-verified attendees can review
Capacity exceeded	Show Sold out and offer waitlist
429 rate limit	Show a wait message and disable the button briefly
Network error	Show retry banner; keep entered form data

4.16 Performance, Accessibility and Responsiveness

- Route-level code splitting so that the 3D module and chart libraries load only when needed.
- Images served at the right size, lazy loaded and with fixed aspect ratios to avoid layout shift.
- Query caching and background refresh so that returning to a page is instant.
- Skeleton loaders instead of blank screens, and optimistic updates for upvotes.
- Lighthouse targets of 90 or more for performance, accessibility and best practices on a mid-range phone profile.
- Testing on narrow widths (360 px), tablets and desktops, and in both themes.

4.17 Testing Approach

Table 4.3: Frontend test plan

Level	Tool	Examples
Unit	Vitest	Countdown hook, time-offset correction, form validators, price formatting
Component	React Testing Library	Team modal steps, QR pass states, feedback gating message
Contract	Mock Service Worker	Responses for success and each error code from the API spec

Level	Tool	Examples
End to end	Playwright	Register, login, join team, pay (test mode), open pass, view certificate
Visual and responsive	Browser device emulation	360 px phone, tablet, desktop; dark and light themes
Performance	Lighthouse, frame profiler	Page speed score; 3D FPS on a mid-range phone

4.18 Deployment Responsibilities

For the frontend I prepare environment-specific settings (API base URL, Razorpay key identifier, feature flags), produce a production build with hashed file names, generate the PWA manifest and service worker, and confirm that the app works when served under the college domain over HTTPS. Ankit Yadav writes the Dockerfile that serves the build through NGINX, the compose file with backend and database, and the CI/CD pipeline. We test the container locally together before the staging deployment in Week 12. Secrets such as payment secrets and database passwords are never placed in the frontend; only the public key identifier is exposed.

4.19 Anticipated Challenges and Planned Solutions

Table 4.4: Challenges and solutions

Challenge	Planned solution
Clock difference between phone and server can make the ring and token disagree	Measure the offset from the server time header; compute remaining time from the server expiry
Weak Wi-Fi at the venue	Prefetch next token early; cache the app shell; show clear offline state; keep the screen awake
Double payment from impatient taps	Disable the button, keep a pending state, rely on server idempotency
API changes breaking the interface	Shared TypeScript types, contract tests with mock responses, interface table updated weekly
3D scene slow on low-end phones	Low-poly models, instancing, pixel-ratio cap, fallback list view
Keeping five members aligned	Weekly meeting, protected main branch, pull-request reviews, issue board

4.20 Integration with Teammates

My work depends on the backend of Garvita Jain (authentication, clubs, events), Ayushi Garg (registration, payments, QR token) and Anshika Pathak (feedback, certificates, points), and shares the application shell and API client with Ankit Yadav. To avoid waiting, I start every module against mock responses that follow the API specification, and switch to the real endpoints when they are ready. Any difference found during integration is recorded in the interface table and fixed in the following week.

CHAPTER 5

14-WEEK WORK PLAN

5.1 My Week-by-Week Plan

Table 5.1 gives my plan for the 14 weeks together with the expected deliverable of each week. The weeks follow the ten phases of the roadmap.

Table 5.1: My 14-week plan

Week	Phase	My tasks	Deliverable / check
1	Setup	Create repository; Vite + React + TypeScript + Tailwind setup; design tokens; wireframes for student screens	Running skeleton app; wireframes reviewed in weekly meeting
2	1	Register and login screens; auth context; token refresh; role-based navbar	User can register and log in against the backend or mock
3	2	Club showcase, category filters, club profile page	All 15 clubs shown with filters
4	3	Event discovery feed, search, tags and filters; event detail page	Filtered feed and detail page with live seat count
5	3	Registration flow UI; team creation and joining with passcode	Solo and team registration working end to end
6	4	My Tickets dashboard; rolling QR pass with countdown ring and watermark	Pass refreshes every 30 s without gaps
7	5	Razorpay checkout integration; payment result screens	Test-mode payment produces a ticket
8	6	Ticket wallet offline cache; PWA install; pass polish	Pass opens quickly on weak network; app installable
9	7	Five-factor feedback form; suggestion board; upvotes	Blocked for unverified users; allowed for verified
10	8	Certificate wallet; public verify page; LinkedIn button; points dashboard	Certificate verification page shows valid status
11	9	Recommended-for-you feed; concierge chatbot UI	Recommendations and chat answers displayed
12	10	3D campus digital twin; beacons; event drawer	Over 50 FPS on a mid-range phone
13	Test	Responsive, dark/light and accessibility audit; Playwright end-to-end tests	Test cases pass; Lighthouse targets met
14	Pilot	User guide; demo video; UI fixes after pilot; final report	Pilot event run; documentation submitted

5.2 Team Overview

Figure 5.1 shows the whole team schedule. In every week each member has one main deliverable, and the weekly meeting reviews all five. The dependencies between my tasks and those of the other four members are listed in Table 5.2.

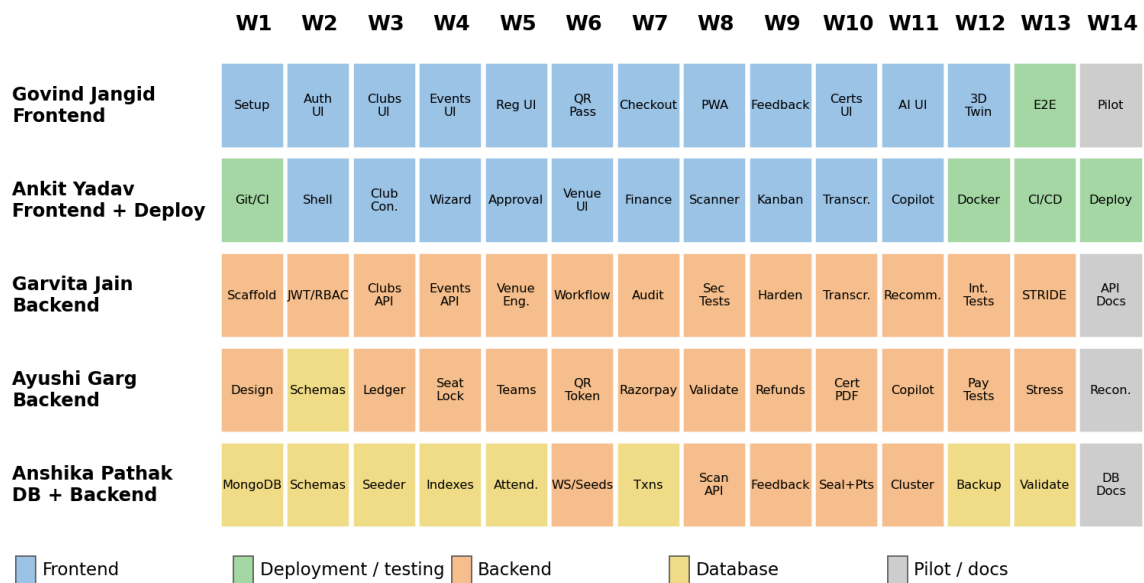


Figure 5.1: 14-week work distribution of the five members

Table 5.2: Dependencies of my weekly tasks on teammates

Week	My task	Depends on
2	Auth screens	Garvita Jain: JWT, refresh rotation and role claims
3-4	Clubs and events UI	Garvita Jain: clubs and events APIs; Anshika Pathak: seeded data
5	Registration and teams	Ayushi Garg: registration and team APIs
6	Rolling QR pass	Ayushi Garg: dynamic QR token endpoint
7	Checkout	Ayushi Garg: Razorpay order and verification endpoints
9	Feedback	Anshika Pathak: feedback gate and suggestion APIs
10	Certificates and points	Anshika Pathak: seal, verify and points APIs; Ayushi Garg: PDF
11	AI feed and chatbot	Garvita Jain: recommender API; Anshika Pathak: chatbot data
12-14	Deployment tests	Ankit Yadav: Docker, NGINX, CI/CD

5.3 Reporting and Review

Every Friday I prepare a short status note listing completed tasks, tasks in progress, blockers and changes to interfaces. The guide reviews progress at the end of key milestones (Weeks 2, 5, 7, 9, 11, 13 and 14). Any slip is handled first by re-ordering tasks within the week and then, if needed, by moving work between sections in agreement with the affected members.

CHAPTER 6

LEARNING OUTCOMES AND SKILLS

6.1 Technical Skills

- Component-based design with React and strict typing with TypeScript.
- Handling server state, caching and optimistic updates with TanStack Query.
- Secure client-side session handling with short-lived access tokens and HttpOnly refresh cookies.
- Building time-sensitive interfaces such as the rolling QR pass, including clock offset correction.
- Integrating a payment gateway safely, with the server as the source of truth.
- Performance-oriented 3D rendering on mobile devices.
- Responsive design, accessibility and testing with Vitest and Playwright.
- Packaging a web application with Docker and NGINX.

6.2 Professional Skills

- Reading and applying a formal specification suite (PRD, SRS, TRD, ERD, API specification).
- Contract-first collaboration between frontend and backend developers.
- Planning work in weekly deliverables and tracking progress with an issue board.
- Writing clear documentation and status reports.

6.3 Leadership and Teamwork

As Team Leader I learned to break a large project into weekly units that fit each member's section, to keep a visible list of interfaces and to resolve disagreements by returning to the specification. I also learned that reviewing a teammate's pull request is a way of learning the whole system and not only my own part.

6.4 Problem Solving and Debugging

Frontend problems often appear at the boundary between systems, such as a token that expires during a form submission or a timer that drifts. The practice of reproducing the problem with a mock, isolating the layer responsible and writing a test that fails before the fix will be applied throughout the project.

6.5 Version Control and Communication

Short-lived branches, small pull requests and meaningful commit messages make collaboration smoother and keep the main branch always deployable. Clear and timely communication with the other four members, including early notice of changes to an interface, is as important as writing good code.

CHAPTER 7

CONCLUSION

This individual report presented my part in the CampusSphere project: the student-facing frontend, my share of deployment, and the role of Team Leader. The work was divided among five members into two frontend and deployment sections and three backend and database sections, and planned over 14 weeks following the ten-phase roadmap so that every member has a clear weekly deliverable.

My modules turn the strong guarantees of the backend into a simple experience for students: a few taps to register and pay, a pass that stays valid by refreshing every 30 seconds, feedback that is open only to verified attendees, certificates that anyone can verify, and a clear view of activity points toward the 100 required by AICTE and RTU. The design keeps the frontend untrusted, type-safe, responsive and testable.

This report documents the design and the plan of my work. The performance and usability targets stated here, such as Lighthouse scores and 50 frames per second, are goals to be measured during Weeks 12 to 14 and during the pilot. The experience of planning the interface in detail, coordinating four teammates and aligning with a formal specification has given me a solid foundation for professional software development and for my future academic and industry projects.

REFERENCES

- [1]AICTE, “Mandatory extracurricular activity points for B.Tech degree,” All India Council for Technical Education, New Delhi, and Rajasthan Technical University, Kota, degree honours ordinances.
- [2]CampusSphere Project Team, “Product Requirements Document (PRD), v1.0.0-PROD,” Arya College of Engineering & I.T., Jaipur, 2026.
- [3]CampusSphere Project Team, “Software Requirements Specification (SRS), IEEE Std 830 format, v1.0.0-PROD,” 2026.
- [4]CampusSphere Project Team, “Technical Requirements Document (TRD), v1.0.0-PROD,” 2026.
- [5]CampusSphere Project Team, “REST API Specification, v1.0.0-PROD,” 2026.
- [6]CampusSphere Project Team, “Security Architecture and Fraud Prevention Specification, v1.0.0-PROD,” 2026.
- [7]CampusSphere Project Team, “Engineering Development Roadmap, v1.0.0-PROD,” 2026.
- [8]D. M’Raihi, S. Machani, M. Pei and J. Rydell, “TOTP: Time-based one-time password algorithm,” IETF RFC 6238, May 2011.
- [9]M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, May 2015.
- [10]Meta Platforms, “React documentation.” [Online]. Available: <https://react.dev/>
- [11]Microsoft, “TypeScript documentation.” [Online]. Available: <https://www.typescriptlang.org/docs/>
- [12]Vite Contributors, “Vite documentation.” [Online]. Available: <https://vite.dev/>
- [13]Tailwind Labs, “Tailwind CSS documentation.” [Online]. Available: <https://tailwindcss.com/docs>
- [14]TanStack, “TanStack Query documentation.” [Online]. Available: <https://tanstack.com/query/latest>
- [15]Razorpay, “Payment gateway and webhook documentation.” [Online]. Available: <https://razorpay.com/docs/>
- [16]Three.js Authors, “Three.js: JavaScript 3D library.” [Online]. Available: <https://threejs.org/>
- [17]Mozilla Developer Network, “Progressive web apps.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

- [18]Playwright Contributors, “Playwright documentation.” [Online]. Available: <https://playwright.dev/>
- [19]Docker Inc., “Docker documentation.” [Online]. Available: <https://docs.docker.com/>